追加カード2枚の多入力ANDプロトコルの安全性検証と効率化

寺本 優斗[1,a)]　縫田 光司[2,b)]

概要：プロトコルの実行に必要な「追加カードの枚数」と「シャッフル回数」はカードベース暗号におけるプロトコルの効率性を表す重要な指標であり，これらの削減はカードベース暗号の分野において非常に重要な研究対象である．その中でも，追加カード数が2枚の n 入力 AND プロトコルのシャッフル回数についての研究は近年数多く行われている．本稿では，SCIS 2026 において本間と水木が提案したプロトコルの修正及び改善を行い，追加カード数が2枚の n 入力 AND プロトコルのシャッフル回数のさらなる削減を実現する．

キーワード：カードベース暗号，秘密計算，AND プロトコル

Security Verification and Efficiency Improvement of Multi-Input AND Protocols Using Two Additional Cards

Yuto Teramoto[1,a)]　Koji Nuida[2,b)]

Abstract: The“number of additional cards”and the“number of shuffles”required to execute a protocol are key metrics for evaluating the efficiency of card-based cryptographic protocols, and reducing these is a critical research focus in the field of card-based cryptography. In particular, there has been a significant amount of research in recent years on the number of shuffles required for n-input AND protocols with two additional cards. In this paper, we demonstrate that it is possible to further reduce the number of shuffles in n-input AND protocols requiring two additional cards by modifying and improving the protocol proposed by Homma and Mizuki at SCIS2026.

Keywords: Card-based cryptography，Secure computation，The number of shuffles

1. はじめに

物理的なカード組を用いて秘密計算を行う手法は**カードベース暗号**と呼ばれる．カードベース暗号では，裏面は ? で区別ができず，表面が ♣ か ♡ のカードが多く用いられており，これらのカードを用いて ♣ ♡ =0, ♡ ♣ =1 と符号化を行いブール値を表現する．$\underbrace{\boxed{?}\boxed{?}}_{x}$ が x の**コミットメント**であるとは，この裏向きの2枚が上記のルールで $x \in \{0,1\}$ の値を表している場合を指す．n 個のコミットメントを入力とし，その論理積を計算するプロトコルを n 入力 AND プロトコルという．また，プロトコルの中で出力がコミットメントの形を取るものをコミット型，コミットメントの形を取らないものを非コミット型と呼ぶ．2入力 AND プロトコルについて，コミット型で代表的なものには2009年に Mizuki と Sone が提案したもの?があり，非コミット型で代表的なものには1989年に den Boer が提案した”Five-Card Trick”?がある．

[1] 九州大学大学院数理学府
Graduate School of Mathematics, Kyushu University
[2] 九州大学マス・フォア・インダストリ研究所
IMI, Kyushu University
[a)] teramoto.yuto.577@s.kyushu-u.ac.jp
[b)] nuida@imi.kyushu-u.ac.jp

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

1.1 追加カード 2 枚での多入力 AND プロトコル

追加カード 2 枚という条件下でのコミット型 n 入力 AND プロトコルにおけるシャッフル回数の削減についての研究は数多く行われている．CANS 2023 において Yoshida らは Mizuki-Sone のプロトコルに Batching 技術を組み合わせたプロトコル群を考案した?．その後，SCIS 2024 では石崎と品川が一般化パイルスクランブルシャッフルとブランチング技術という新たな技術を導入して更なるシャッフル回数の削減を行った?．また，SCIS 2026 において本間と水木は小泉らのコミットメント半減 AND プロトコルと Kuzuma らの多入力 AND プロトコルを使用することによって既存の手法よりもシャッフル回数を削減することに成功している?.

1.2 本稿の貢献

本稿では SCIS 2026 において本間と水木によって提案された追加カード 2 枚の多入力 AND プロトコルについて，安全性が崩れている部分についての修正を行うと同時に Batching 技術の識別用カードの選び方の更なる改善を行う．具体的には識別用カードの選び方を工夫することでガベージコミットメントをより有効に活用できるようにする．これらを行うことによって，本間と水木によって提案された追加カード 2 枚の多入力 AND プロトコルの安全性を確保することに成功し，シャッフル回数についてもわずかであるが改善された.

1.3 本稿の構成

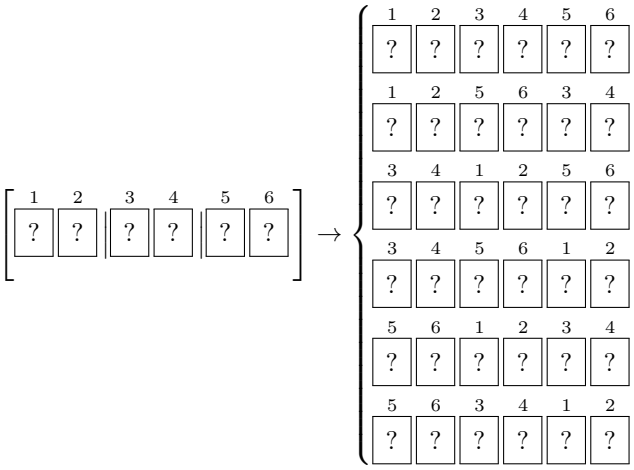
以下，本稿の構成は次の通りである．2 節で準備を与え，3 節で既存研究を紹介する．その後 4 節で提案手法を紹介し，改善したプロトコルを示す．最後に 5 節で結論を述べる.

2. 準備

本節では 3 節以降で説明するプロトコルに用いる基本的な技術としてパイルスクランブルシャッフルと Batching 技術を紹介する.

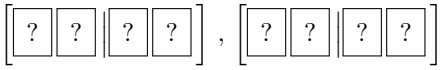
2.1 パイルスクランブルシャッフル

あるカード列を同じ枚数のカードからなる複数の束に分け，それらの順番を一様ランダムに並べ替えるシャッフルを**パイルスクランブルシャッフル** (以下 PSS と表記する) と呼ぶ. この際，各束内のカードの順序については保持される．PSS において束の数が 2 つであるものは特に**ランダム二等分割カット** (以下 RBC と表記する) と呼ばれる．カード列に PSS を適用するときは，$[\cdot \mid \cdot \mid \cdots \mid \cdot]$ のように表記する．例として 6 枚のカード列を 2 枚ずつの 3 つの束に分けて PSS を適用すると次のようになる.



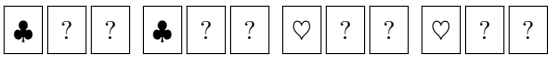
2.2 Batching 技術について

Batching 技術とは 2020 年に Shinagawa と Nuida?によって提案された手法であり，識別用の追加カードを用いることで複数の PSS を 1 つの PSS に結合する技術である．Batching 技術を用いることでプロトコルにおけるシャッフル回数の削減を図ることができる．2 つの RBC

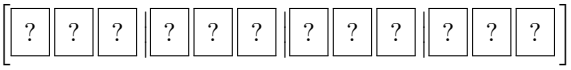


を 1 つの PSS で行う Batching 技術の具体例について説明する.

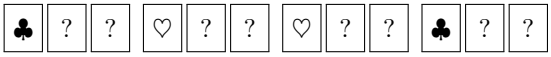
(1) 2 つの RBC のそれぞれの束の先頭に識別用のカード列（以下識別子と呼ぶ）として ♣ と ♡ を挿入する.



(2) 識別子を裏返し，4 つの束に対して PSS を適用する.



(3) 識別子として追加したカードをオープンする.



(4) ♣ の束が左，♡ の束が右に来るように並べ替え，識別子を取り除くことで 2 つの RBC が 1 つの PSS で行えたことになる.

♣=0, ♡=1 として識別子を 2 進数のビット列と捉えることができるため，一般に $2k\lceil \log_2 k \rceil$ 枚の追加カードがあれば k 個の RBC を Batching 技術を用いて 1 つの PSS にまとめることができる.

3. 既存研究

3.1 小泉らのコミットメント半減 AND プロトコル

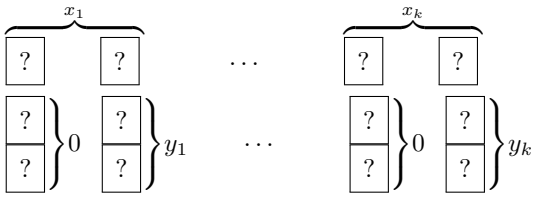
小泉らのコミットメント半減 AND プロトコル?は $x_1, \cdots, x_k, y_1, \cdots, y_k \in \{0, 1\}$ を入力として k 個の論理

© 2026 Information Processing Society of Japan

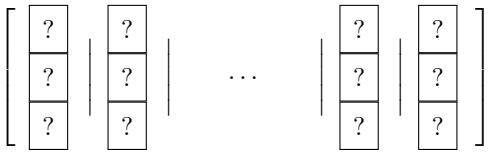
積 $x_1 \wedge y_1, \cdots, x_k \wedge y_k \in \{0,1\}$ を出力するコミット型プロトコルである．しかし，追加カードが $2k$ 枚必要であることと，出力のどのコミットメントがどの論理積に対応するのか分からないことを注意しておく．

以下，小泉らのコミットメント半減 AND プロトコルの具体的な手順を説明する．

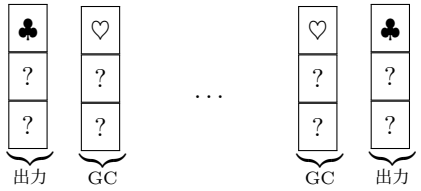
(1) 下のようにコミットメントを配置する．



(2) 縦に並んだ 3 枚を 1 つの束とみなし，$2k$ 個の束に対して PSS を実行する．



(3) 1 段目の $2k$ 枚のカードをオープンする．♣ の下の 2 枚が $x_1 \wedge y_1, \cdots, x_k \wedge y_k$ のコミットメントになっている．



プロトコル終了時に表向きになっているカードは**フリーカード**として回収でき，別のプロトコルでそのまま追加カードとして用いることができる．多くの場合フリーカードの ♣ と ♡ の割合は等しくなる．以下，フリーカード ♣ ♡ の組をフリーペアと呼ぶ．また，プロトコル終了時に裏向きで残っているコミットメントは**ガベージコミットメント**として回収される．ガベージコミットメントはシャッフルを 1 回適用することで別のプロトコルでフリーカードとして用いることが可能になる．（ガベージコミットメントに対してシャッフルを適用してフリーペアとして回収する操作を**ガベージコレクション**と呼ぶ．）

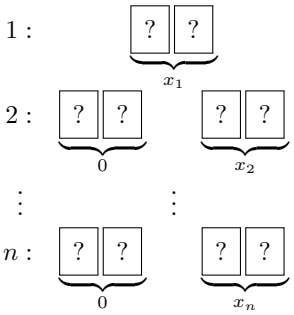
小泉らのコミットメント半減プロトコルではプロトコル実行後に $2k$ 枚のフリーカードと k 個のガベージコミットメントが手に入る．

3.2 Kuzuma らの多入力 AND プロトコル

Kuzuma らの多入力 AND プロトコルは $x_1, \cdots, x_n \in \{0,1\}$ を入力として $x_1 \wedge \cdots \wedge x_n \in \{0,1\}$ を出力するコミット型プロトコルである．後の説明のためにまずは安全性を考慮していないプロトコルとして手順 A について述べる．

手順 A

(1) 下のように $x_1, \cdots, x_n$ のコミットメント及び追加カードを並べる．



(2) 1 行目のコミットメントをオープンし，その値を v とする．また，$i \coloneqq 2$ とする．

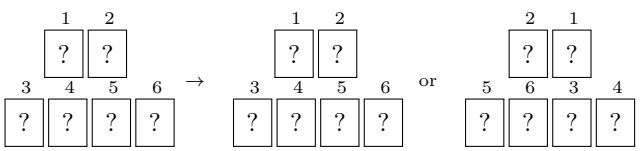
(3) $v = 0$ の場合は i 行目の左側のコミットメントをオープンする．$v = 1$ の場合は i 行目の右側のコミットメントをオープンする．その後 v をオープンしたコミットメントの値に書き換える．

(4) $i \coloneqq i + 1$ として，$i \leq n - 1$ の場合は (3) に戻る．

(5) $v = 0$ の場合は n 行目の左側のコミットメントを出力とし，$v = 1$ の場合は n 行目の右側のコミットメントを出力とする．

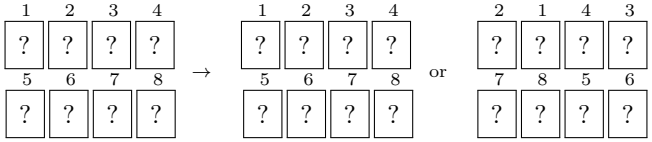
上記の手順 A を行うことによって $x_1 \wedge \cdots \wedge x_n \in \{0,1\}$ のコミットメントを得ることができる．しかし，$x_1, x_1 \wedge x_2, \cdots, x_1 \wedge x_2 \wedge \ldots \wedge x_{n-1}$ の値が漏洩してしまっているため最初に述べたように手順 A は秘密計算になっていない．そのため，秘密計算にするために次のシャッフルを導入する．

1 行目と 2 行目に対して次のようなシャッフルを考える．



このようなシャッフルを $\mathcal{S}_1$ と呼ぶ．

また，i 行目と $i+1$ 行目に対して次のようなシャッフルを考える．



© 2026 Information Processing Society of Japan

このようなシャッフルを $\mathcal{S}_i$ と呼ぶ．ここで，$\mathcal{S}_i(1 \le i \le n-1)$ について各 $\mathcal{S}_i$ は 1 回の PSS で実行できることと手順 A の出力結果に影響しないことを注意しておく．

手順 A と $\mathcal{S}_1, \mathcal{S}_2, \cdots, \mathcal{S}_{n-1}$ を用いてコミット型 n 入力 AND プロトコルを次のように構成する．

(1) 手順 A(1) のようにカードを並べる．

(2) $\mathcal{S}_1, \mathcal{S}_2, \cdots, \mathcal{S}_{n-1}$ のうち $\mathcal{S}_1, \mathcal{S}_3, \mathcal{S}_5 \cdots$ は互いに素なのでこれらを Batching を用いて 1 回の PSS で実行する．

(3) $\mathcal{S}_1, \mathcal{S}_2, \cdots, \mathcal{S}_{n-1}$ のうち $\mathcal{S}_2, \mathcal{S}_4, \mathcal{S}_6 \cdots$ は互いに素なのでこれらを Batching を用いて 1 回の PSS で実行する．

(4) 得られた配置に対して手順 A を適用する．これによって得られた出力が $x_1 \wedge \cdots \wedge x_n$ のコミットメントになっている．

3.3 本間と水木の n 入力 AND プロトコル

本間と水木が SCIS 2026 で提案した n 入力 AND プロトコル?について説明する．以下のプロトコル群において a, b, g はそれぞれ，残コミットメント数，フリーペア数，ガベージコミットメント数を表している．

(1) $(a, b, g) \coloneqq (n, 1, 0)$ とする．

(2) a 個の残コミットメント，b 個のフリーペア，g 個のガベージコミットメントがある状況で次のいずれかを実行する．

(a)
$a \ge 2k$ かつ $b \ge k$ を満たす最大の k に対して小泉らのコミットメント半減 AND プロトコルを適用する．このとき (a, b, g) は以下のように変化する．

$$(a, b, g) \coloneqq (a - k, b, g + k)$$

(b-i)
$\left\lfloor \frac{k}{2} \right\rfloor$ が偶数，または $\left\lceil \log_2 \left\lfloor \frac{k}{2} \right\rfloor \right\rceil$ が偶数であり，$a \ge k$ かつ $b \ge k + \left\lfloor \frac{k}{2} \right\rfloor \left\lceil \log_2 \left\lfloor \frac{k}{2} \right\rfloor \right\rceil$ を満たすとき，Kuzuma らの k 入力 AND プロトコルを適用する．このとき (a, b, g) は以下のように変化する．

$$(a, b, g) \coloneqq (a - k + 1, b, g + k - 1)$$

(b-ii)
$\left\lfloor \frac{k}{2} \right\rfloor$ が奇数，かつ $\left\lceil \log_2 \left\lfloor \frac{k}{2} \right\rfloor \right\rceil$ が奇数であるとする．このとき，$g \ge 1$ ならばガベージコミットメントを 1 つ識別子として用いることができるため，f を次のように定義する．

$$f = \begin{cases} 0 & (g = 0) \\ 1 & (g = 1) \end{cases}$$

$a \ge k$ かつ $b \ge k + \left\lfloor \frac{k}{2} \right\rfloor \left\lceil \log_2 \frac{k}{2} \right\rceil - f$ を満たすとき，Kuzuma らの k 入力 AND プロトコルを適用する．このとき (a, b, g) は以下のように変化する．

$$(a, b, g) \coloneqq (a - k + 1, b - 1 + f, g + k - f)$$

(c)
g 個のガベージコミットメントに対して，ガベージコレクションを適用する．（以下 g-GC と表記する．）このとき (a, b, g) は以下のように変化する．

$$(a, b, g) \coloneqq (a, b + g, 0)$$

(3) $a \ge 2$ であれば, ステップ (2) へ戻る．

上記のステップ (2) における具体的な選択を決めることで，1 つの具体的なプロトコルを定めることができる．表 **??**に動的計画法を用いて求めた最適な選択をした場合のこのプロトコルのシャッフル回数 $(1 \le n \le 500)$ がまとめられている．

表 1 本間と水木の提案プロトコルにおけるシャッフル回数

入力数 n	シャッフル回数	入力数 n	シャッフル回数
2	1	28–37	11
3	2	38–53	12
4	3	54–71	13
5	4	72–107	14
6–7	5	108–140	15
8–9	6	141–193	16
10–12	7	194–291	17
13–15	8	292–404	18
16–19	9	405–500	19
20–27	10		

4. 提案手法

4.1 本間と水木のプロトコルの安全性の崩れとその修正

先ほどのプロトコル群の (b-i) においてフリーペアは全部で $k + \left\lfloor \frac{k}{2} \right\rfloor \left\lceil \log_2 \left\lfloor \frac{k}{2} \right\rfloor \right\rceil$ 個必要であるとされていた．これの内訳を詳しく書くと，$k-1$ 個は Kuzuma らの AND プロトコルの際に並べる 0 コミットメント用，1 個は Batching の際に各束の枚数を等しくするためにシャッフル $\mathcal{S}_1$ に追加する用，$\left\lfloor \frac{k}{2} \right\rfloor \left\lceil \log_2 \left\lfloor \frac{k}{2} \right\rfloor \right\rceil$ 個は Batching の識別子のカード用である．(b-i) と (b-ii) での (a, b, g) の変化から，本間と水木のプロトコルでは $\mathcal{S}_1$ に追加したカードをフリーカードとして回収していると考えられるが，これをフリーカードとして回収してしまうと $\mathcal{S}_1$ の情報が漏れてしまう．具

© 2026 Information Processing Society of Japan

体的にはフリーペアが追加したときと同じ状態でオープンされた場合は $\mathcal{S}_1$ では元のカード列がそのまま得られており，フリーペアが追加したときとは反対の状態でオープンされた場合は $\mathcal{S}_1$ では並べ替えが起こったカード列が得られているとわかる．Kuzuma らの多入力 AND プロトコルにおいて $\mathcal{S}_1$ の情報が漏れることは x_1 の情報が漏れることと同義であるため秘密計算が実現できていないと言える．

そのため，$\mathcal{S}_1$ に追加したフリーペアはガベージコミットメントとして回収しなければならない．しかし，そもそも $\mathcal{S}_1$ に追加したカードは公開する必要がないコミットメントになるため，ガベージコミットメントが存在する場合はガベージコミットメントを $\mathcal{S}_1$ に追加してガベージコミットメントとして回収するという手法も可能である．このようにすることでプロトコルの安全性を確保できると同時にガベージコミットメントの活用によるシャッフル回数の削減も期待できる．

4.2 Batching における識別用カードの選択方法

本間と水木は最下位ビットをオープンしない識別子を用いることで常に $\boxed{\clubsuit}$ と $\boxed{\heartsuit}$ の割合が等しい識別用のカードの選び方を提案した．本節ではその手法を元にして，ガベージコミットメントを有効に活用することによって必要なフリーカードの枚数を削減できる識別用のカードの選び方を提案する．

$l = \lceil \log_2 k \rceil$ とする．つまり 1 束あたりに挿入する識別用のカードの枚数を l とする．また，$\boxed{\clubsuit}$ を 0，$\boxed{\heartsuit}$ を 1 と対応づけてカード列を l ビットの 2 進数として解釈する．

1 つの RBC を形成する 2 つの束に対して a を表すビット列と b を表すビット列を識別用のカード列として用いる場合, その識別子を $\begin{cases} a \\ b \end{cases}$ と表記する. ここで $a = b$ のときは追加したカードを全てオープンする識別子であり，a が偶数かつ $b = a + 1$ のときは最下位ビットをオープンしない識別子である．

以下，RBC 1 つにつき合計 l 個のフリーペアとガベージコミットメント（$\boxed{\clubsuit}$ と $\boxed{\heartsuit}$ の割合は等しい）を用いて k 個の RBC を一つの PSS にまとめる Batching における識別用のカードの選び方，すなわち全体として合計 lk 個のフリーペアとガベージコミットメントを用いる識別子の選び方を述べる．

A．k が偶数のとき

（1） $2^l \geq 2i + (k - i)$ を満たす最大の整数 i を求める．これは具体的に求まり，$i = 2^l - k$ である．

（2） 0〜$i - 1$ のビット列について $\begin{cases} 0 \\ 1 \end{cases}, \begin{cases} 2 \\ 3 \end{cases}, \cdots, \begin{cases} i-2 \\ i-1 \end{cases}$ を識別用のカード列として選ぶ．

（3） $2^l - i$ 〜 $2^l - 1$ のビット列において $\begin{cases} 2^l - 2 \\ 2^l - 1 \end{cases}, \begin{cases} 2^l - 4 \\ 2^l - 3 \end{cases}, \cdots, \begin{cases} 2^l - i \\ 2^l - i + 1 \end{cases}$ を識別用のカード列として選ぶ．

（4） 残りの i 〜 $2^l - i - 1$ 全てのビット列を $\begin{cases} i \\ i \end{cases}, \begin{cases} i+1 \\ i+1 \end{cases}, \begin{cases} i+2 \\ i+2 \end{cases}, \cdots, \begin{cases} 2^l - i + 1 \\ 2^l - i + 1 \end{cases}$ として識別用のカード列として選ぶ．

B．k が奇数かつ l が奇数のとき

（1） $2^l \geq 2i + (k - i)$ を満たす最大の整数 i を求める．これは具体的に求まり，$i = 2^l - k$ である．

（2） $\begin{cases} 2^{\frac{l+1}{2}} - 2 \\ 2^{\frac{l+1}{2}} - 1 \end{cases}$ を識別用のカード列として選ぶ．

（3） $\begin{cases} 2^l - 2^{\frac{l+1}{2}} \\ 2^l - 2^{\frac{l+1}{2}} \end{cases}$ と $\begin{cases} 2^l - 2^{\frac{l+1}{2}} + 1 \\ 2^l - 2^{\frac{l+1}{2}} + 1 \end{cases}$ を識別用のカード列として選ぶ．

（4） $\begin{cases} 0 \\ 1 \end{cases}, \begin{cases} 2 \\ 3 \end{cases}$ $\cdots$ と順に $\dfrac{i-1}{2}$ 個を識別用のカード列として選ぶ．（ただし，$\begin{cases} 2^{\frac{l+1}{2}} - 2 \\ 2^{\frac{l+1}{2}} - 1 \end{cases}$ が含まれている場合は飛ばす．）

（5） $\begin{cases} 2^l - 2 \\ 2^l - 1 \end{cases}, \begin{cases} 2^l - 4 \\ 2^l - 3 \end{cases}$ $\cdots$ と順に $\dfrac{i-1}{2}$ 個を識別用のカード列として選ぶ．（ただし，$\begin{cases} 2^l - 2^{\frac{l+1}{2}} \\ 2^l - 2^{\frac{l+1}{2}} + 1 \end{cases}$ が含まれている場合は飛ばす．）

（6） 用いていない全てのビット列を $\begin{cases} a \\ a \end{cases}$ の形の識別用のカード列として選ぶ．

C．k が奇数かつ l が偶数のとき

（1） $2^l \geq 2i + (k - i)$ を満たす最大の整数 i を求める．これは具体的に求まり，$i = 2^l - k$ である．

（2） $\begin{cases} 2^{\frac{l}{2}} - 1 \\ 2^{\frac{l}{2}} - 1 \end{cases}$ を識別用のカード列として選ぶ．

（3） $\begin{cases} 2^{\frac{l}{2}} - 2 \\ 2^{\frac{l}{2}} - 2 \end{cases}$ と $\begin{cases} 2^l - 2^{\frac{l}{2}} + 1 \\ 2^l - 2^{\frac{l}{2}} + 1 \end{cases}$ を識別用のカード列として選ぶ．

© 2026 Information Processing Society of Japan

(4) $\begin{Bmatrix}0\\1\end{Bmatrix}, \begin{Bmatrix}2\\3\end{Bmatrix}, \cdots$ と順に $\frac{i-1}{2}$ 個を識別用のカード列として選ぶ.（ただし, $\begin{Bmatrix}2^{\frac{l}{2}}-2\\2^{\frac{l}{2}}-1\end{Bmatrix}$ が含まれる場合は飛ばす.）

(5) $\begin{Bmatrix}2^l-2\\2^l-1\end{Bmatrix}, \begin{Bmatrix}2^l-4\\2^l-3\end{Bmatrix}, \cdots$ と順に $\frac{i-1}{2}$ 個を識別用のカード列として選ぶ.（ただし, $\begin{Bmatrix}2^l-2^{\frac{l}{2}}\\2^l-2^{\frac{l}{2}}+1\end{Bmatrix}$ が含まれる場合は飛ばす.）

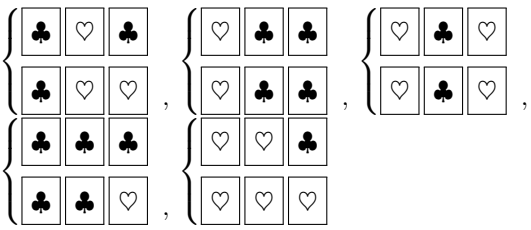
(6) $2^l-2^{\frac{l}{2}}$ を除く用いてないビット列全てを $\begin{Bmatrix}a\\a\end{Bmatrix}$ の形の識別用のカード列として選ぶ.

このように識別子を選ぶことにより k が偶数のとき，または k が奇数かつ l が奇数のときは 2^l-k 個，k が奇数かつ l が偶数のときは 2^l-k-1 個のガベージコミットメントを有効に活用することが可能になる.

4.3 $k=5$ のときの Batching の実行方法の具体例

$k=5$ の場合の具体的な識別子の選び方, 及び Batching の適用手順を示す,

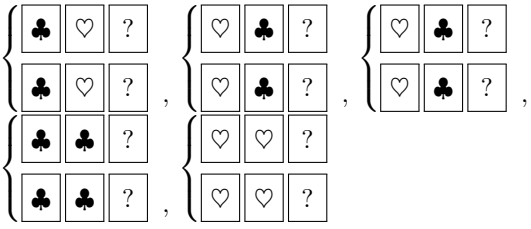
(1) 識別子として $\begin{Bmatrix}2\\3\end{Bmatrix}, \begin{Bmatrix}4\\4\end{Bmatrix}, \begin{Bmatrix}5\\5\end{Bmatrix}, \begin{Bmatrix}0\\1\end{Bmatrix}, \begin{Bmatrix}6\\7\end{Bmatrix}$ を選ぶ.
すなわち,



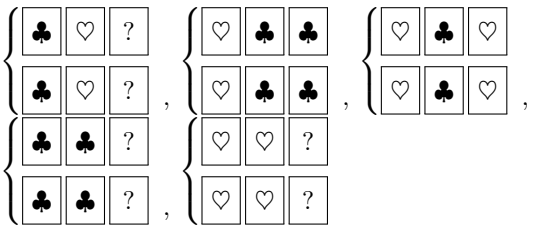
のカード列を識別子として用いる.

(2) 各束の先頭に識別子を挿入し PSS を実行する.

(3) 識別子の最下位ビット以外をオープンする.



(4) (3) でオープンしたもののうち, 同じカード列が 4 つ存在するものについては残り 1 枚もオープンする.



(5) 識別子にしたがって束を並べ替える. 最下位ビットが裏向きの束についても同じ情報の束は 2 つしか存在しないため識別が可能である.

最後までオープンされなかった部分にガベージコミットメントを利用したと仮定した場合，12 個のフリーペアと 3 個のガベージコミットメントを用いて $k=5$ の場合の Batching を行うことができた.

4.4 提案プロトコル

ここまでに提案した手法を用いて本間と水木のプロトコル群を改良する. 具体的には Kuzuma らの k 入力 AND プロトコルが用いられている (b-i) と (b-ii) を以下の (b') に書き換える.

(b')
$l=\left\lceil \log_2 \left\lfloor \frac{k}{2} \right\rfloor \right\rceil$ とする. また,

$$d = \begin{cases} 2^l - \lfloor \frac{k}{2} \rfloor - 1 & (\lfloor \frac{k}{2} \rfloor \text{ が奇数かつ } l \text{ が偶数}) \\ 2^l - \lfloor \frac{k}{2} \rfloor & (\lfloor \frac{k}{2} \rfloor \text{ が偶数または } l \text{ が奇数}) \end{cases}$$

$h=\min(g, d+1)$ とおく.
$a \geq k$ かつ $b \geq k + \left\lfloor \frac{k}{2} \right\rfloor l - h$ を満たすとき Kuzuma らの k 入力 AND プロトコルを適用する. このとき (a,b,g) は以下のように変化する.

$$(a,b,g) \coloneqq (a-k+1, b-(d+1)+h, g+k+d-h)$$

ガベージコミットメントを活用できる部分には最大限ガベージコミットメントを利用している．もし，必要なガベージコミットメントの数 ($d+1$ 個) に対して実際に存在するガベージコミットメントが不足している場合はガベージコミットメントの枠にフリーペアを追加してガベージコミットメントとして回収することを想定している.

4.5 具体例

$n=72$ の場合の今回の提案プロトコルの具体例について述べる．初期状態は $(a,b,g) \coloneqq (72,1,0)$ である.

(1) 3.3 節の手順 (a) を $k=1$ で 2 回適用する．このとき $(72,1,0) \to (70,1,2)$ と遷移する.

(2) 2-GC を適用する．このとき $(70,1,2) \to (70,3,0)$ と遷移する.

(3) 手順 (a) を $k=3$ で 2 回適用する．このとき

© 2026 Information Processing Society of Japan

$(70,3,0) \to (64,3,6)$ と遷移する．

(4) 6-GC を適用する．このとき $(64,3,6) \to (64,9,0)$ と遷移する．

(5) 手順 (a) を $k = 9$ で 2 回適用する．このとき $(64,9,0) \to (46,9,18)$ と遷移する．

(6) 18-GC を適用する．このとき $(46,9,18) \to (46,27,0)$ と遷移する．

(7) 手順 (a) を $k = 23$ で 1 回適用する．このとき $(46,27,0) \to (23,27,23)$ と遷移する．

(8) 手順 (a) を $k = 11$ で 1 回適用する．このとき $(23,27,23) \to (12,27,34)$ と遷移する．

(9) 4.4 節の手順 (b') を $k = 12$ で 1 回適用する．このとき $(12,27,34) \to (1,27,45)$ と遷移する．

以上より $n = 72$ の場合のシャッフル回数は 13 回である．SCIS 2026 で提案された本間と水木のプロトコルのシャッフル回数は 14 回であったため，シャッフル回数を削減することに成功している．

4.6 最小シャッフル回数について

今回の提案プロトコル群におけるシャッフル回数が最小のプロトコルを動的計画法を用いて計算し，SCIS2026 で提案された本間と水木のプロトコルのシャッフル回数と比較する．

3.3 節で示したプロトコルの (b-i) と (b-ii) を (b') に書き換えたプロトコルを実行し，初期状態 $(a,b,g) \coloneqq (n,1,0)$ から計算対象の残コミットメント数が x，フリーペア数が y になったとする．このとき $(a,b,g) \coloneqq (x,y,n-x-y+1)$ になっている．これを実現する最小のシャッフル回数を $s_n(x,y)$ と書くことにする．3.3 節の手順 (a) で選択される k の最大値を k_a とし，4.3 節の手順 (b') で選択される k の最大値を k_b とする．$s_n(x,y)$ を求める動的計画法の疑似コードをアルゴリズム 1 に示す．

表**??**が動的計画法を用いて今回の提案プロトコル群を用いた最小シャッフル回数を調べた結果である．本間と水木のプロトコルにおけるシャッフル回数を示した表**??**とここで示した表**??**を比較すると，大きな変更は行っていないため大半の n で同じ結果が得られているが，$n = 20, 21$ の場合と $n = 72, 73, 74$ の場合において今回の提案プロトコルは本間と水木のプロトコルよりもシャッフル回数を削減することができていることが分かる．本間と水木のプロトコルは既存手法の中でもシャッフル回数が最小のプロトコルの 1 つであったため，このことから今回の提案プロトコルは従来手法と比較してもシャッフル回数を削減できたと言える．

表 2 提案プロトコルにおけるシャッフル回数

入力数 n	シャッフル回数	入力数 n	シャッフル回数
2	1	28–37	11
3	2	38–53	12
4	3	54–74	13
5	4	75–107	14
6–7	5	108–140	15
8–9	6	141–193	16
10–12	7	194–291	17
13–15	8	292–404	18
16–21	9	405–500	19
22–27	10		

Algorithm 1: 最小シャッフル数の探索

```
n ← 入力数;
for 1 ≤ x ≤ n do
    for 1 ≤ y ≤ n − 1 do
        S[x][y] ← ∞;
    end
end
S[n][1] ← 0;
for 1 ≤ y ≤ n − 1 do
    for n + 1 − y ≥ x ≥ 2 do
        if S[x][y] + 1 < S[x − k_a][y] then
            S[x − k_a][y] ← S[x][y] + 1;
        end
        if S[x][y] + 2 < S[x − (k_b + 1)][y − d − 1 + h] then
            S[x − k_b + 1][y − d − 1 + h] ← S[x][y] + 2;
        end
        if S[x][y] + 1 < S[x][n − x + 1] then
            S[x][n − x + 1] ← S[x][y] + 1;
        end
    end
end
```

5. 結論

本稿では本間と水木が提案した追加カード 2 枚の n 入力 AND プロトコルの修正および改善を行った. ガベージコミットメントの有効な活用を考えることによって安全性を確保すると同時にシャッフル回数もわずかではあるが削減することができた.

RBC を結合する Batching の識別子の選び方について今回はガベージコミットメントの最大利用を優先しフリーペアが減る可能性を許容したが, フリーペアが減らない程度にガベージコミットメントを活用したいという場面も考えられるため今後そのような識別子の選び方も含め Batching 技術の改良について幅広く検討を進めていく. また, ガベージコミットメントの有効活用を考えていく事は他のプロトコルの改善にも繋がると思われるため, 今後更なる研究を進めていこうと考えている.

© 2026 Information Processing Society of Japan

謝辞　本研究は，JST 経済安全保障重要技術育成プログラム【JPMJKP24U2】の支援を受けたものです.

参考文献

[1] B. den Boer：“More efficient match-making and satisfiability The Five Card Trick”，Advances in Cryptology – EUROCRYPT 1989，LNCS vol.434，pp.208-217，Springer，Berlin，Heidelberg，1990.

[2] 本間 耀，水木 敬明：“追加カード 2 枚の多入力 AND 秘密計算のさらなるシャッフル回数削減”，暗号と情報セキュリティシンポジウム 2026 (SCIS 2026)，2A3-2，2026.

[3] 石崎悠斗, 品川和雅：“追加カード 2 枚の多入力 and 計算におけるシャッフル回数の新しい削減方法”，2024 年暗号と情報セキュリティシンポジウム（SCIS 2024)，no.3D1-1，pp.1–8，2024.

[4] 小泉 康一，水木 敬明，花岡 悟一郎：“カードベース秘匿マッチングと多入力 AND 秘密計算”，コンピュータセキュリティシンポジウム 2024 (CSS 2024)，3H5-2，2024.

[5] T. Kuzuma, R. Isuzugawa, K. Toyoda, D. Miyahara, T. Mizuki: “Card-based Single-shuffle Protocols for Secure Multiple-input AND and XOR Computations”，Proceedings of the 9th ACM ASIA Public-Key Cryptography Workshop (APKC 2022)，pp.51-58，ACM，NY，2022.

[6] T. Mizuki，H. Sone：“Six-card secure AND and four-card secure XOR”，Frontiers in Algorithmics，LNCS vol.5598, pp.358–369, Springer, Berlin, Heidelberg, 2009.

[7] K. Shinagawa, K. Nuida：“A single shuffle is enough for secure card-based computation of any Boolen circuit”，Discrete Applied Mathematics，vol.289，pp.248–261，2021.

[8] T. Yoshida, K. Tanaka, K. Nakabayashi, E. Chida, T. Mizuki：“Upper Bounds on the Number of Shuffles for Two-Helping-Card Multi-Input AND Protocols”，Cryptology and Network Security (CANS 2023)，LNCS vol.14342，pp.211-231，Springer，Singapore，2023.

© 2026 Information Processing Society of Japan